BigBird: Transformers for Longer Sequences

Architecture, Motivation, Key Innovations and Comparison with BERT and Longformer

Based on Zaheer et al., Google Research, NeurIPS 2020 (arXiv:2007.14062)

1. Overview

BigBird is a Transformer encoder that replaces the full self-attention of BERT with a **sparse attention** mechanism. It reduces the memory and compute cost of attention from quadratic to **linear** in sequence length, so that sequences of roughly 4,096 tokens (about 8x what a standard BERT-style model handles on similar hardware) can be processed. The authors also prove that this sparse model keeps two key theoretical properties of full attention: it is a **universal approximator** of sequence functions and it is **Turing complete**.

2. Motivation: the Quadratic Bottleneck

In standard self-attention every token attends to every other token. For a sequence of length n, the attention matrix has n x n entries, so time and memory both grow as $O(n^2)$. In practice this limits models such as BERT to about 512 tokens. Long inputs (full articles, books, legal or scientific papers, multi-document question answering, genomic sequences) must therefore be truncated or split into chunks, which loses cross-chunk context and complicates the pipeline.

The goal of BigBird is to keep the benefits of Transformers while making attention cheap enough for long inputs, without discarding the theoretical expressiveness that makes full attention powerful.

3. Architecture and Sparse Attention Mechanism

BigBird keeps the usual Transformer encoder stack (multi-head attention, feed-forward layers, residual connections and layer norm). Only the attention pattern changes. Viewing tokens as nodes in a graph, each token attends only to a small, fixed-size set of neighbours instead of all n tokens. This set is built from three components:

3.1 Random attention

Each query attends to r randomly chosen keys. Random edges make the attention graph behave like a sparse random graph, where information can travel between any two tokens in a small (logarithmic) number of hops. This gives fast global information flow even though each token only looks at a few others.

3.2 Sliding window (local) attention

Each token attends to w neighbouring tokens (for example w/2 on each side). Language is strongly local, so nearby words carry most of the needed context. Local windows capture this locality and, as in convolutions, stacking layers widens the effective receptive field.

3.3 Global tokens

A small number g of tokens attend to the entire sequence and are attended to by every token. They act as hubs that summarise and redistribute information, similar to [CLS]. BigBird proposes two ways to choose them:

- **ITC (Internal Transformer Construction):** some existing tokens (such as [CLS]) are made global.
- **ETC (Extended Transformer Construction):** extra dedicated global tokens are added to the sequence (for example for question tokens or sentence-level summaries), which gives more flexibility.

3.4 Block-sparse implementation

Sparse, token-level gather operations are slow on GPUs and TPUs, which are optimised for dense matrix multiplication. BigBird therefore groups tokens into **blocks** (for example, blocks of 64 tokens) and applies the window, random and global patterns at the block level. Each query block attends to a few neighbouring key blocks, a few random key blocks and the global blocks. This turns sparse attention into a small number of dense block multiplications, which is what makes the linear-complexity attention fast in practice.

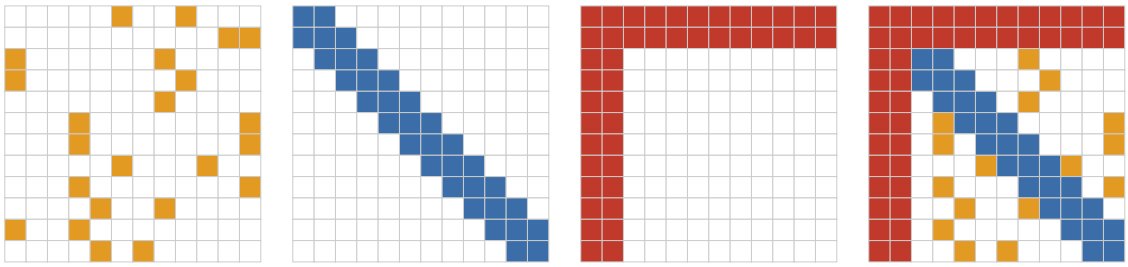

Figure 1. Illustrative attention patterns (rows = queries, columns = keys). Red = global, blue = window, orange = random. The combined pattern is the union of the three.

Because w, r and g are constants that do not depend on n, each token attends to O(1) others, so the total cost is O(n). For encoder-decoder tasks such as summarisation, BigBird applies sparse attention in the encoder (long input) and keeps full attention in the decoder, since output sequences are short.

4. Key Innovations

- **Linear-complexity attention** that combines random, window and global components in one pattern.
- **Theoretical guarantees:** proof that the sparse model is a universal approximator of sequence functions and Turing complete, and analysis showing why global tokens matter.
- **Random attention** as a principled way to get short paths between all token pairs (graph-theoretic motivation).

- **Block-sparse hardware-friendly implementation** that gives real speed and memory gains on accelerators.
- **Applications beyond NLP**, notably genomics (DNA) where long-range context is important.

5. Computational Complexity: BERT vs Longformer vs BigBird

Aspect	BERT (full attention)	Longformer	BigBird
Attention per token	All n tokens	Sliding window (w) + task-specific global tokens	Window (w) + random (r) + global (g)
Time / memory	$O(n^2)$	O(n x w), linear in n	O(n x (w + r + g)), linear in n
Typical max length	512 tokens	About 4,096 tokens	About 4,096 tokens
Global information	Direct (everything sees everything)	Via global tokens and stacked windows	Via global tokens, random edges and stacked windows
Random links	No	No	Yes
Theory	Universal approximator, Turing complete	Mainly empirical	Proven universal approximator and Turing complete

Key point: Longformer and BigBird are both linear in sequence length, so the practical difference is not asymptotic cost. The differences are BigBird's added random connections, its theoretical analysis, its block-sparse implementation, and the way global tokens are used (ITC/ETC). With fixed constants, doubling the sequence length roughly doubles the attention cost for BigBird, whereas for BERT it roughly quadruples it.

6. Suitable Applications for Long Documents

- **Long-document question answering** (for example Natural Questions, HotpotQA, TriviaQA, WikiHop), where the answer evidence may be far from the question or spread over several passages.
- **Abstractive summarisation** of long articles, scientific papers (arXiv, PubMed) and patents using BigBird-Pegasus.
- **Long-document classification** such as legal contracts, medical records and reports.
- **Information extraction and retrieval** over long passages or multiple concatenated documents.
- **Genomics:** DNA sequence modelling, promoter-region and chromatin-profile prediction, where long context helps.

7. Performance Characteristics

In the paper, BigBird pre-trained with masked language modelling (initialised from RoBERTa checkpoints) and fine-tuned on longer inputs gave improvements over models limited to 512 tokens on long-context question answering benchmarks, and BigBird-Pegasus gave state-of-the-art

summarisation results at the time on long-document datasets. On genomics tasks, longer-context pre-training improved downstream results. Gains are largest when relevant information really lies beyond 512 tokens; on short inputs BigBird performs similarly to BERT/RoBERTa. In common implementations (for example Hugging Face), when the input is short enough the model can fall back to full attention, since block-sparse attention brings no benefit there. Specific benchmark scores vary by dataset and version, so please consult the original paper for exact numbers.

8. Advantages and Limitations

Advantages	Limitations
Linear memory and time in sequence length	Still bounded: practical lengths are a few thousand tokens, not unlimited
Handles about 8x longer inputs than BERT on similar hardware	Sparse attention is an approximation; some tasks need many more layers than full attention (the paper shows such a case)
Retains universal approximation and Turing completeness	Constant overhead is large: block-sparse code is more complex and, for short inputs, slower than or equal to full attention
Hardware-friendly block structure	Random attention is less interpretable and adds some nondeterminism to the pattern; hyperparameters (block size, r, w, g) need tuning
Flexible global tokens (ITC/ETC) for task structure	Pre-training on long sequences is costly; very long inputs (books) still need hierarchical or retrieval methods
Works for text and non-text sequences such as DNA	Newer long-context models and efficient kernels (such as FlashAttention) have narrowed its advantage

9. Conclusion

BigBird shows that a carefully designed sparse pattern, combining local windows, a few global tokens and random links, can replace full attention, bringing cost down from $O(n^2)$ to $O(n)$ while keeping the theoretical power of Transformers. It makes Transformers practical for long documents and genomic sequences and remains an influential reference design for efficient attention.

Reference

Zaheer, M., Guruganesh, G., Dubey, A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., Ahmed, A. (2020). Big Bird: Transformers for Longer Sequences. Advances in Neural Information Processing Systems (NeurIPS). arXiv:2007.14062.